Advanced Quantitative Text Analysis

Day 2: From Concepts to Labels

Petro Tolochko & Noëlle Lebernegg

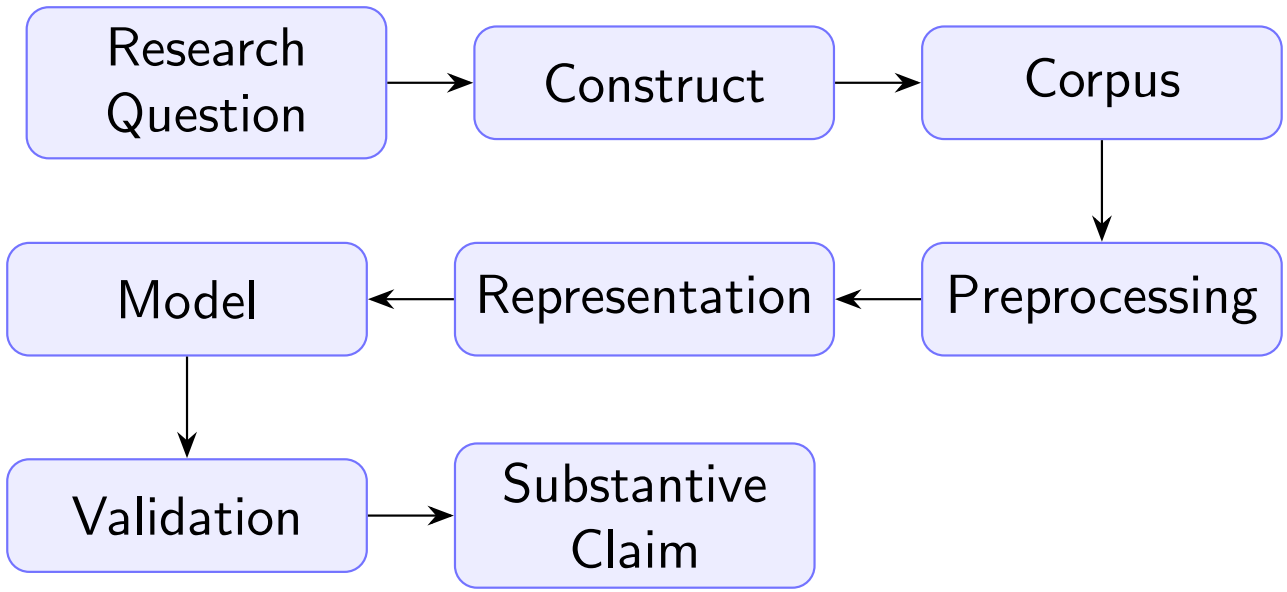
Yesterday and Today

- **Yesterday:** how do we represent text?
  - tokens
  - document-term matrices
  - bag-of-words
  - TF-IDF
  - similarity and distance
- **Today:** how do we assign meaning to those representations?
  - rule-based baselines
  - human annotation
  - supervised classification
  - validation and error analysis

Today

1. From concepts to labels
2. Dictionaries and rule-based measurement
3. Human annotation and codebooks
4. Supervised learning: features, labels, and baselines
5. Authorship attribution and Naive Bayes
6. SMS classification and other models
7. Evaluation, validation, and practical comparison

The Text Analysis Pipeline



Today: how constructs become labels, rules, features, and models.

From Concepts to Labels

- Text analysis often starts from concepts that are not directly observed:
  - sentiment
  - populism
  - framing
  - incivility
  - uncertainty
  - misinformation
  - ideology
  - complexity
- We need to map text onto those concepts.
- That mapping is a **measurement decision**.

Three Ways to Assign Meaning to Text

1. **Rule-based methods**
   - researcher writes the rules
   - dictionaries, regex, search strings
2. **Human annotation**
   - humans label examples using a codebook
3. **Supervised learning**
   - model learns patterns from labeled examples

Key point

All three are measurement strategies. None is automatically valid.

Dictionaries

- Rule-based method
- List of words or phrases that indicate a category
- Can be:
  - existing and validated
  - adapted from previous work
  - created for a specific project

Use Cases for Dictionaries

1. **Classification**
   - Is this text about topic X?
   - Is this sentence positive or negative?
2. **Regression / scoring**
   - How much uncertainty language appears?
   - How negative is this article?
3. **Search strings**
   - Which documents enter the corpus?

Dictionary for Classification

Let D^+ be a set of positive words and D^- a set of negative words. Given a text $t = (w_1, \ldots, w_n)$:

$$\text{pos}(t) = \sum_{i=1}^{n} \mathbf{1}[w_i \in D^+], \qquad \text{neg}(t) = \sum_{i=1}^{n} \mathbf{1}[w_i \in D^-]$$

$$\hat{y}(t) = \begin{cases} \text{positive}, & \text{pos}(t) > \text{neg}(t) \\ \text{negative}, & \text{neg}(t) > \text{pos}(t) \\ \text{neutral}, & \text{otherwise.} \end{cases}$$

Dictionary for Scoring

Each word w has an associated weight $s(w)$.

$$s(\text{“excellent”}) = +3, \qquad s(\text{“terrible”}) = -3$$

Score of a text $t = (w_1, \ldots, w_n)$:

$$\text{score}(t) = \sum_{i=1}^{n} s(w_i)$$

Length-normalized score:

$$\hat{s}(t) = \frac{\text{score}(t)}{n}$$

Normalization

- Raw counts can be misleading: longer texts usually have higher counts.
- Common normalizations:

$$\hat{s}_D(t) = \frac{s_D(t)}{|t|} \qquad \text{or} \qquad \hat{s}_D(t) = \frac{s_D(t)}{|t|} \times 1000$$

- Always ask: what denominator makes sense for the claim?

Dictionary Strengths

- Transparent
- Easy to explain
- Easy to reproduce
- Often useful with small data
- Good when concepts are lexically explicit
- Useful as baselines for more complex models

Dictionary Weaknesses

- Context-insensitive
- Vulnerable to polysemy
- Misses synonyms and paraphrases
- Sensitive to domain and language
- Negation is difficult
- May confuse word prevalence with concept prevalence

False Positives and False Negatives

Term	Intended concept	Possible problem
attack	conflict	"heart attack"
green	environment	colour / Green Party
risk	uncertainty	technical finance vocabulary
people	populism	generic word
fake	misinformation	quotation / denial / discussion of the term

Regex as Controlled Pattern Matching

- Regular expressions specify search patterns.
- Useful for:
  - word variants
  - dates
  - percentages
  - URLs
  - hashtags
  - legal references
  - search strings
- But regex is brittle and easy to overcomplicate.

Regex Examples

Pattern	Matches
`migrat\w+`	migration, migrant, migrating
`colo(u)?r`	color, colour
`\d+(\.\d+)?\%`	5%, 12.5%
`#[A-Za-z0-9_]+`	hashtags
`https?://\S+`	URLs

Rule-Based Methods as Weak Supervision

- Rule-based methods can create noisy labels.
- These labels can be used to:
  - identify candidate positives
  - bootstrap annotation
  - train preliminary classifiers
  - prioritize human review
- This is not “less advanced”.
- It is often how scalable pipelines begin.

From Rules to Human Labels

Labels Are Not Ground Truth From Heaven

- Supervised models learn from labels.
- Labels come from:
  - human annotation
  - institutional categories
  - metadata
  - search strings
  - platform labels
  - previous models
- Label quality defines the ceiling of model quality.

Concepts and Data

- What do we actually want to measure?
- Where do we find it?
- What textual evidence indicates the concept?
- Who decides?
- How do we know whether the decision is valid?

Human Labels: Codebook

- A codebook is the tool used to code content.
- It defines how texts should be mapped to categories or scores.
- It should be detailed enough that:
  - you can apply it again in the same way
  - other people can apply it in the same way
  - disagreements can be interpreted

What a Good Codebook Contains

- Definition of the concept
- Unit of annotation
- Inclusion criteria
- Exclusion criteria
- Positive examples
- Negative examples
- Edge cases
- Decision rules

Annotation Is Measurement

- Human annotation is not ground truth from heaven.
- It is an operationalization of a concept.
- Annotators can disagree because:
  - the concept is vague
  - the text is ambiguous
  - the codebook is incomplete
  - the task is genuinely difficult

Reliability vs. Validity

Reliability	Do coders apply the coding scheme consistently?
Validity	Does the coding scheme capture the intended concept?

Important

Reliable nonsense is still nonsense.

Intercoder Reliability

- Common reliability measures:
  - Percent agreement
  - Cohen's κ
  - Krippendorff's α
- Useful, but not sufficient.
- High agreement can reflect an easy but invalid task.
- Low agreement may reveal a concept that needs refinement.

Creating Labeled Data

1. Define the concept.
2. Write a codebook.
3. Sample texts for annotation.
4. Train coders.
5. Annotate independently.
6. Assess reliability.
7. Resolve disagreements or model them.
8. Split data for model training and evaluation.

From Label to Classification

Machine Learning Paradigms

Supervised learning	**Unsupervised learning**
outcome variable defined	no predefined outcome
labeled data	unlabeled data
focus on prediction	focus on pattern discovery
classification or regression	clustering, topics, scaling

Supervised Text Classification

- Input: text represented as features
- Output: label
- Model learns a mapping:

$$f(X) \longrightarrow y$$

- X = document features
- y = label
- f = learned classifier

Dictionary vs. Supervised Machine Learning

Dictionary-based methods	**Supervised ML**
researcher writes rules	learns from labeled examples
transparent	more flexible
no training data required	requires annotated data
brittle	can capture complex patterns
limited context sensitivity	may learn shortcuts

Dictionaries encode theory directly; supervised ML learns from labeled data.

Feature Representation

- Supervised models do not consume raw text directly.
- They consume **features**.
- Common feature representations:
  - raw word counts
  - TF-IDF
  - n-grams
  - dictionary scores
  - embeddings
  - transformer representations
- Representation choices from Day 1 directly affect classification.

Machine Learning 101

- Model:
$$Y = f(X; \theta)$$
- Objective function:
$$L(\theta; D)$$
- Optimization:
$$\widehat{\theta} = \arg\min_{\theta} L(\theta; D)$$

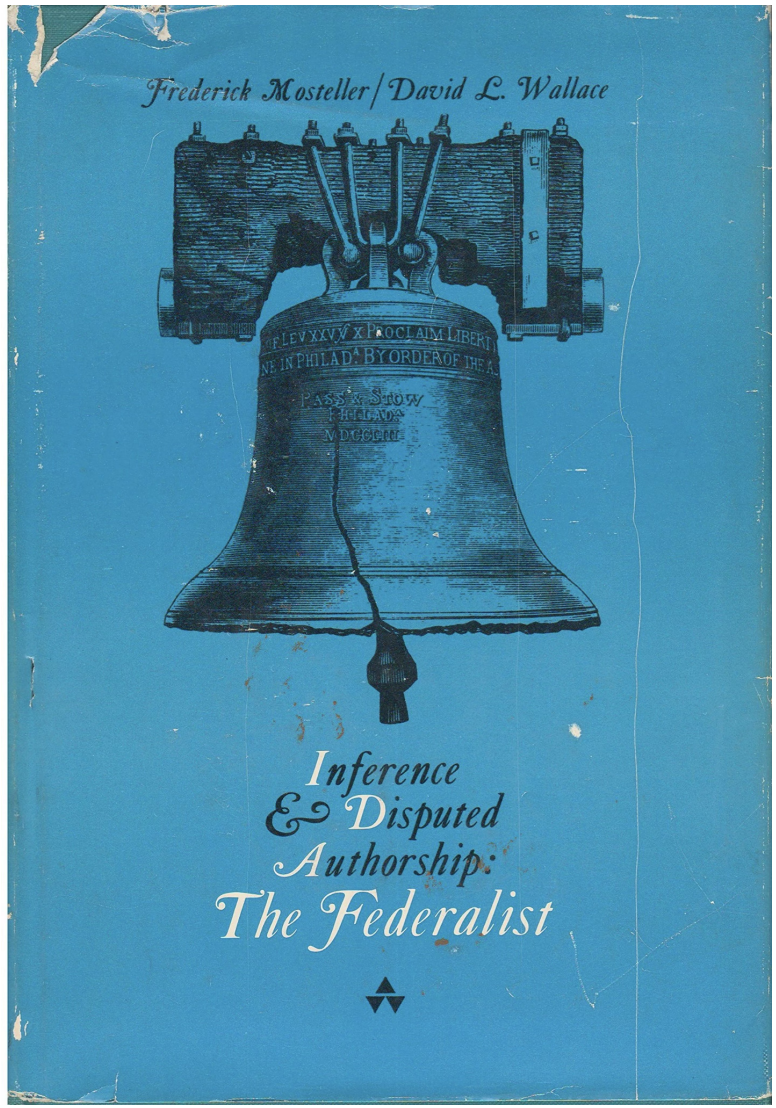
The model learns to minimize a loss on labeled examples.

Baselines Matter

- Always start with simple baselines.
- Examples:
  - majority-class baseline
  - dictionary baseline
  - logistic regression with TF-IDF
  - Naive Bayes
- A complex model should earn its complexity.
- If a transformer barely beats a dictionary, that is a result.

Example: From Word Use to Authorship

Inference and Disputed Authorship



The Federalist Papers

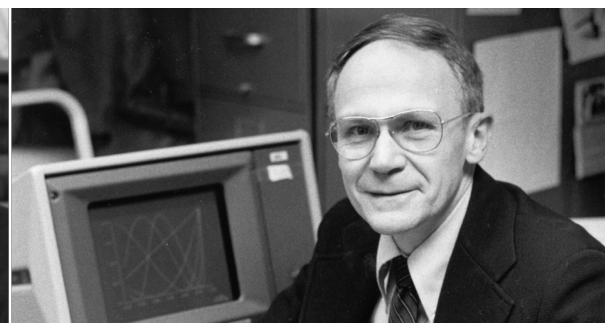
- 85 essays published in 1787–88 advocating ratification of the U.S. Constitution
- Written under the pseudonym **Publius**
- Main authors:
  - Alexander Hamilton
  - James Madison
  - John Jay
- Historical mystery: who wrote which of the disputed essays?

The Authorship Puzzle

- 73 essays: authorship fairly certain
- 12 essays: disputed between Hamilton and Madison
- Historians debated authorship for more than 150 years
- Ideal test case for quantitative text analysis:
  - known authors
  - disputed documents
  - repeated writing by same authors
  - stylistic traces in word use

Mosteller & Wallace

- One of the classic early statistical studies of text
- Used Bayesian methods and early computing power
- Showed that statistical evidence can resolve a historical authorship debate
- Important lesson: sometimes "unimportant" words are highly informative

Why Function Words?

- In many text applications, we remove stop words.
- But in authorship attribution, function words can be useful.
- They are:
  - frequent
  - stylistic
  - difficult to consciously manipulate
  - less topic-dependent than content words
- Day 1 lesson returns: preprocessing depends on the research question.

Dimension Reduction

- Remove the stop words?
- Still too many words.
- Maybe remove all words except selected function words.
- The reduction is not just computational.
- It is also a theory of what distinguishes authors.

Simplified Example

- Focus on three words:
  - “man”
  - “by”
  - “upon”
- The rates with which authors use these words may indicate authorship.

Word Counts

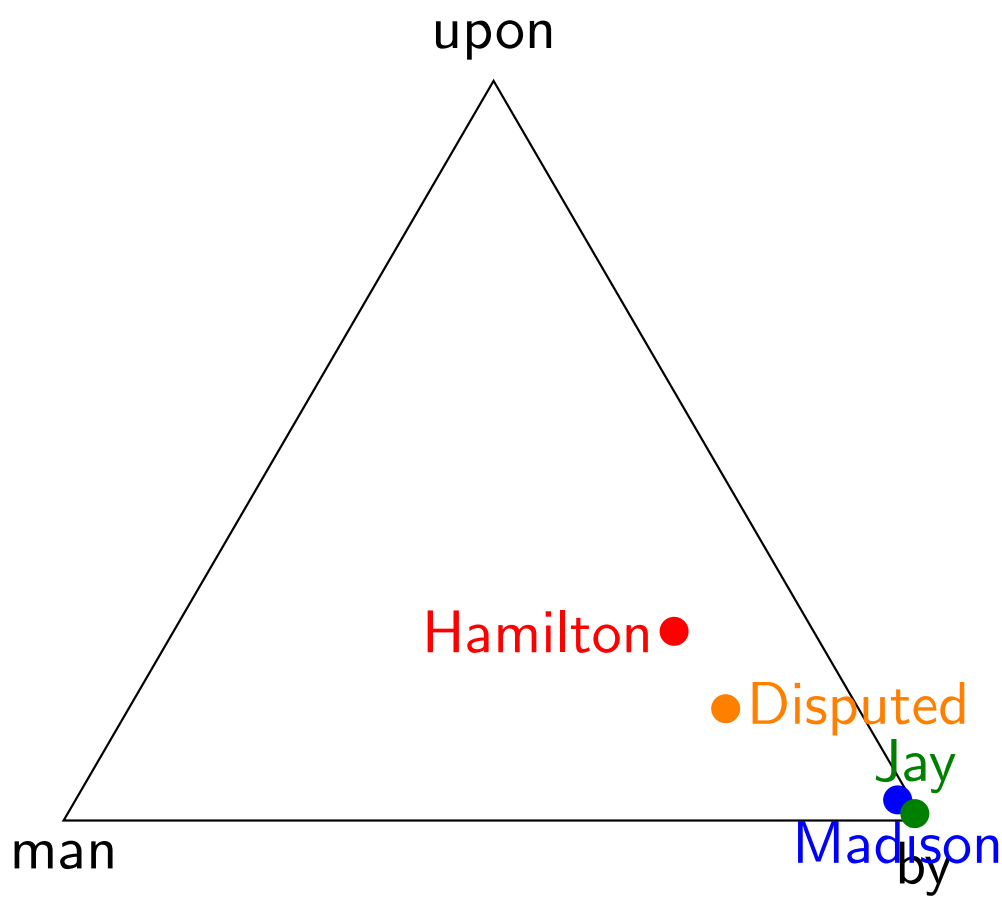
	man	by	upon
Hamilton	102	859	374
Madison	17	474	7
Jay	0	82	1

Word Proportions

	man	by	upon
Hamilton	.076	.643	.280
Madison	.034	.952	.014
Jay	.000	.988	.012

- Each author has a different distribution over words.
- This is the core idea behind a simple language model.

Word Proportions in a Probability Simplex



From Counts to Probabilities with the Multinomial Model

- Start with word counts for each author:

$$\text{Hamilton: } \{\text{man} = 102,\ \text{by} = 859,\ \text{upon} = 374\}$$

- Normalize counts to get probabilities:

$$p(\text{man}) = .076, \quad p(\text{by}) = .643, \quad p(\text{upon}) = .280$$

- These probabilities define the author's distribution over words.

Multinomial Likelihood

- For a new document, we observe counts: $x = (x_{\text{man}}, x_{\text{by}}, x_{\text{upon}})$.
- The probability of seeing x given author A is:

$$P(x \mid A) = \frac{N!}{\prod_i x_i!} \prod_i p_{A,i}^{x_i}$$

- $N = \sum_i x_i$: total words in the document
- $p_{A,i}$: probability of word i for author A

From Likelihood to Classification

- Compute $P(x \mid \text{Hamilton})$ and $P(x \mid \text{Madison})$.
- Compare: which author makes the observed document more likely?
- Classification rule:

$$\hat{A} = \arg \max_{A \in \{\text{Hamilton, Madison, Jay}\}} P(x \mid A)$$

From the Federalist Papers to Naive Bayes

- Mosteller & Wallace used a multinomial model of language.
- This is closely related to the likelihood used in multinomial Naive Bayes.
- Modern text classification often relies on the same idea:

$$\hat{C} = \arg\max_{C} P(C) \prod_{i} P(w_i \mid C)^{x_i}$$

- Key differences today:
  - full vocabularies rather than a few selected words
  - smoothing to handle zero probabilities
  - larger corpora and efficient software implementations

Naive Bayes: Intuition

- Which class is most likely to have generated this text?
- Estimate word probabilities within each class.
- Assign new documents to the class where the words are most probable.

$$P(c \mid d) \propto P(c) \prod_i P(w_i \mid c)$$

The "Naive" Assumption

- Naive Bayes assumes features are conditionally independent given the class:

$$P(X \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

- For text, this means treating word occurrences as independent conditional on the class.
- This is obviously false.
- But it often works surprisingly well for text classification.

Zero Probabilities and Smoothing

- Problem: if a word never appears in a class during training, $P(w_i \mid C) = 0$.
- Then any document containing that word has zero likelihood for that class.
- Solution: smoothing.

$$P(w_i \mid C) = \frac{\text{count}(w_i, C) + \alpha}{\sum_j \text{count}(w_j, C) + \alpha V}$$

- $\alpha = 1$ gives Laplace smoothing.

From Probabilities to Boundaries

Another Example: SMS Spam Detection

Positive class: spam	**Negative class: ham**
unsolicited promotions	ordinary personal messages
prize or lottery messages	coordination and reminders
suspicious links or calls	informal language
high-cost phone numbers	abbreviations and typos

Linear Classifiers

- Many strong text classifiers are linear models.
- Examples:
  - logistic regression
  - linear support vector machines
- Why they work well for text:
  - text features are high-dimensional
  - sparse vectors often become linearly separable
  - regularization controls overfitting

Logistic Regression

- Predicts probability of class membership.
- For binary classification:

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(-x^\top \beta)}$$

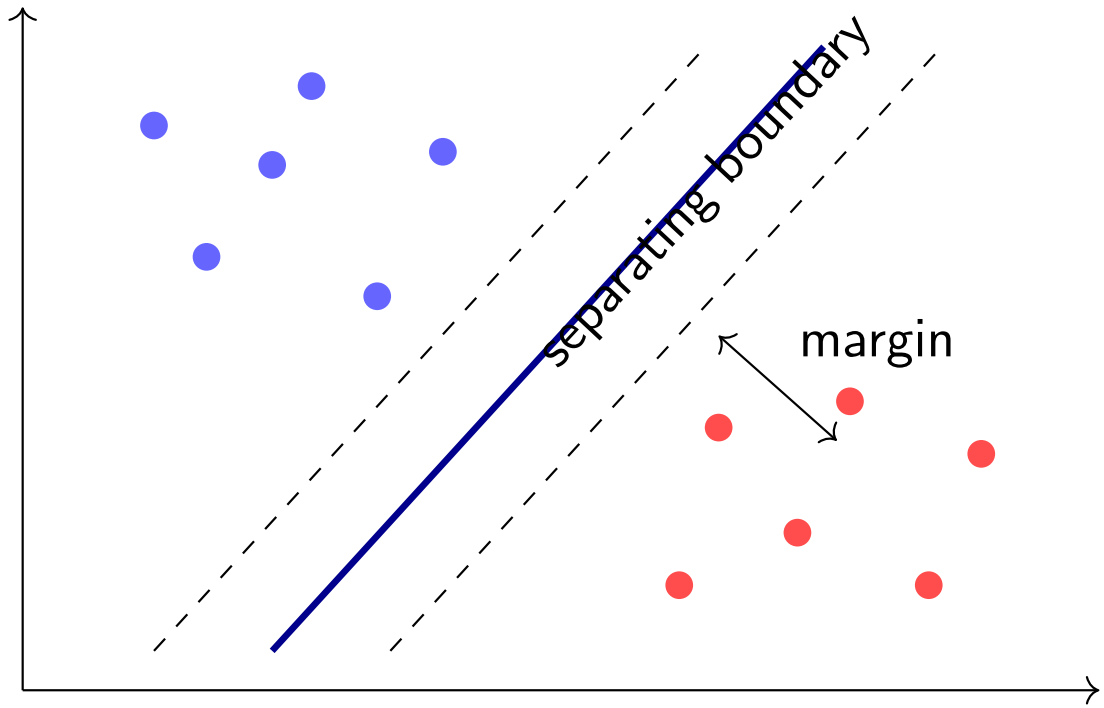
- Useful because:
  - coefficients can be inspected
  - regularization works well with many text features
  - strong baseline for sparse text classification

Support Vector Machines: Intuition

- Represent each document as a feature vector.
- Find a boundary separating classes.
- Choose the boundary with the largest margin.
- Often works well for high-dimensional sparse text data.

$$\widehat{y} = \text{sign}(w \cdot x + b)$$

SVM Margin

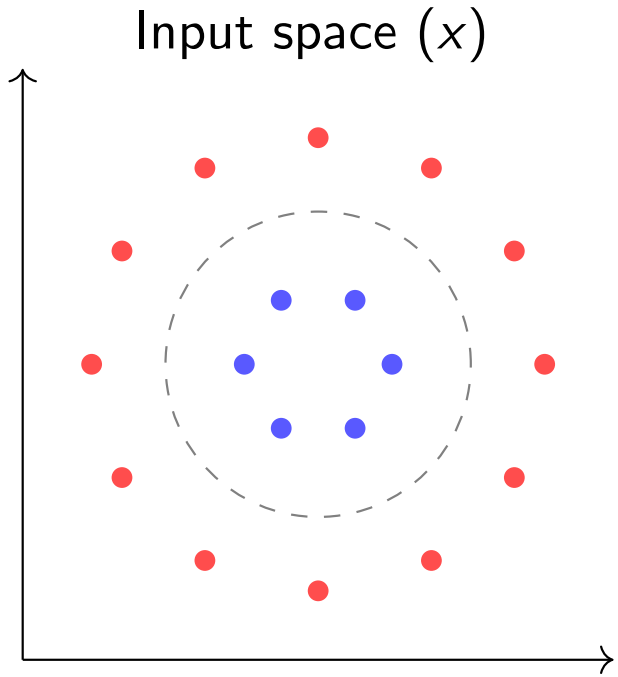


SVM chooses the separating boundary with the largest margin.

Why This Detour?

- We have just seen linear classifiers: logistic regression and SVMs.
- Text vectors are high-dimensional, so linear models are often very strong.
- The geometry helps explain what classifiers mean by a decision boundary.
- The next few slides show why more flexible boundaries sometimes matter.

When Linear Boundaries Fail



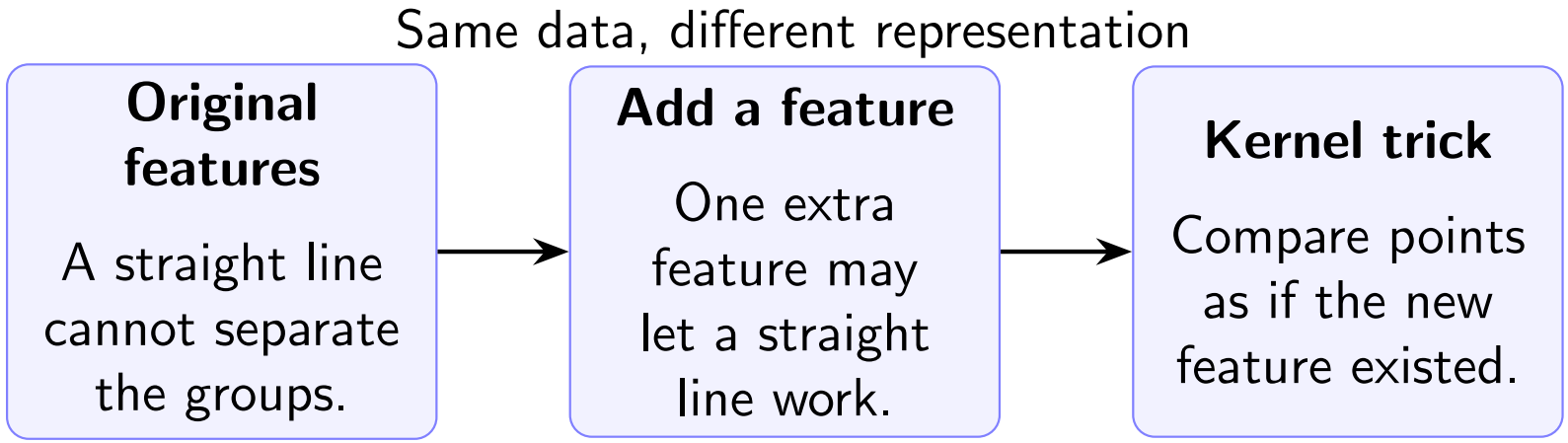
Linear classifiers (like SVMs) can't separate all datasets.

Example: circular data; no straight line can split inner and outer classes.

Need to think beyond linear boundaries.

No linear hyperplane in 2D can separate these points.

From New Features to the Kernel Trick



Remember

Mapping = create new features. Kernel trick = compare in the new representation.

Nonlinear Boundaries and Kernels

- Instead of forcing a linear boundary in the original space, we can transform the data into a higher-dimensional space.
- In that new space, the data may become linearly separable.
- Formally, a mapping:

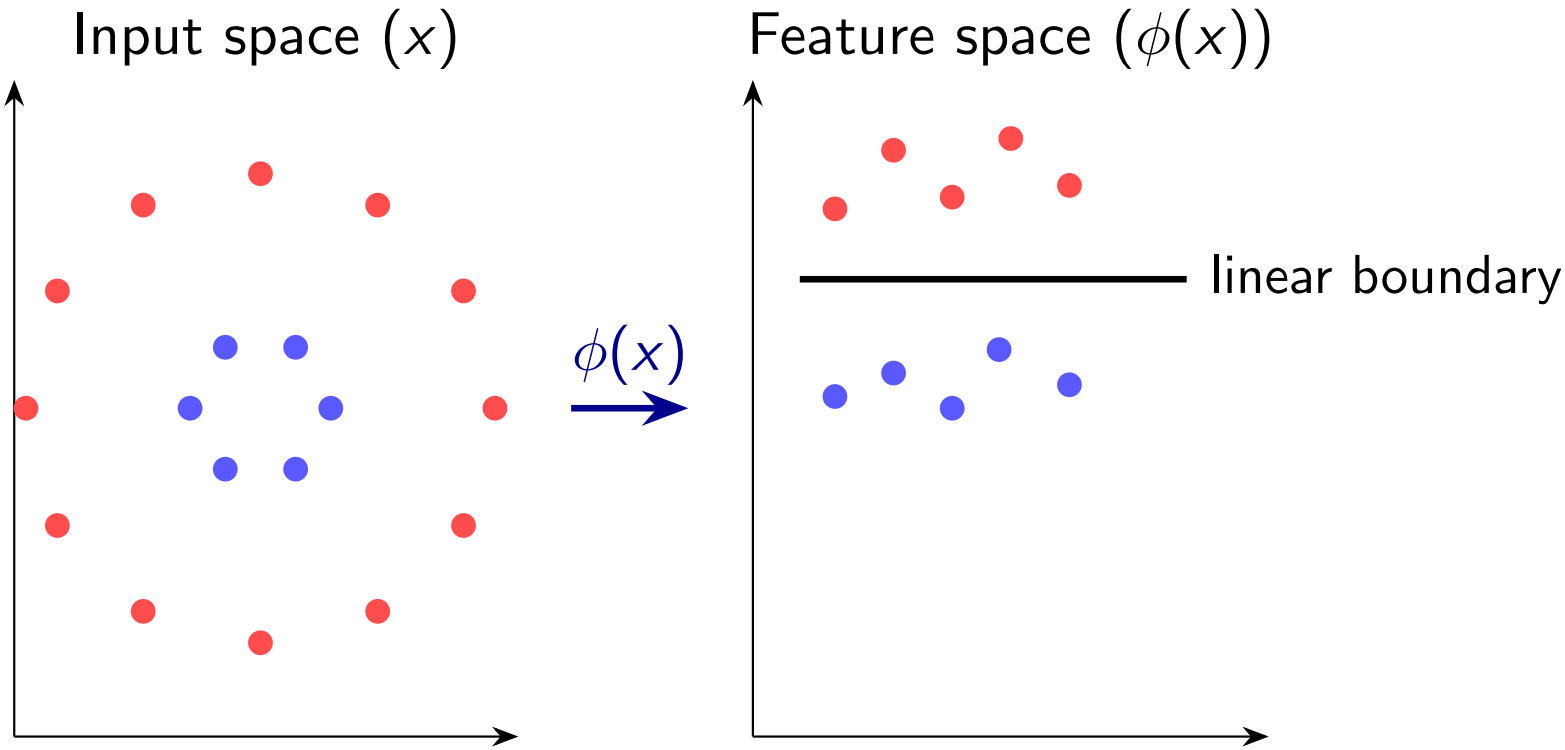
$$\phi : \mathbb{R}^n \longrightarrow \mathbb{R}^m, \qquad m > n$$

- Rather than computing $\phi(x)$ directly, SVMs use kernel functions to compute inner products:

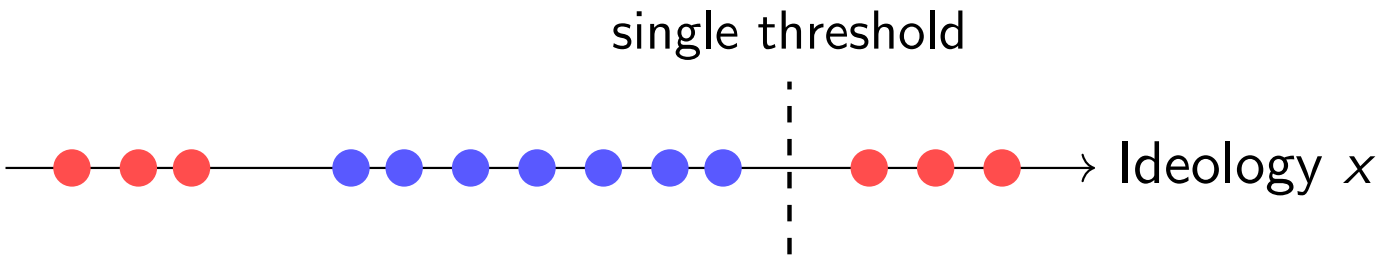
$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

- This is the **kernel trick**: high-dimensional geometry without explicitly computing the transformed coordinates.

The Kernel Trick

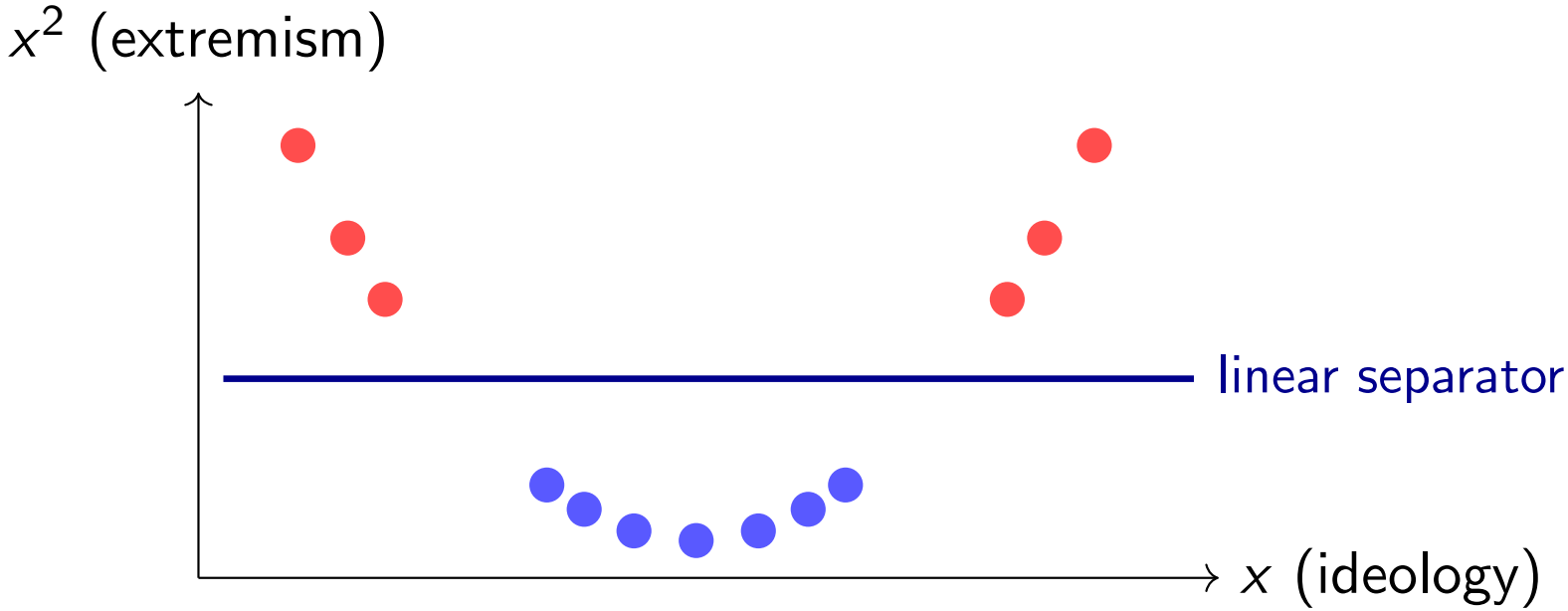


Ideology in 1D: Not Linearly Separable



Extremes (red) lie on both sides of the center; one cut on a line cannot separate them from moderates (blue).

Nonlinear Map $\phi(x) = (x, x^2)$: Now Linearly Separable



After $\phi(x)$, a horizontal linear boundary separates extremes (red) from moderates (blue).

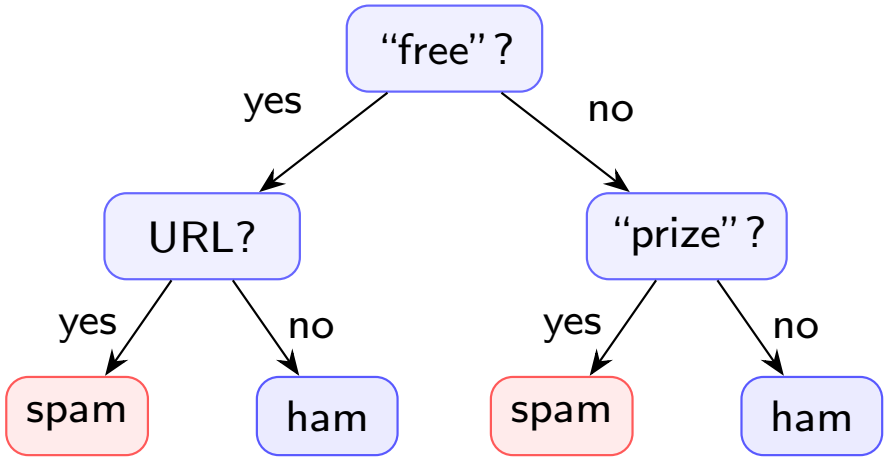
Back to Text

- The geometric examples are intuition, not the main recipe.
- In sparse text classification, linear models with TF-IDF are often hard to beat.
- Kernels and nonlinear models matter when interactions or curved boundaries are substantively plausible.

When Should We Try a Kernel?

- Start with a linear classifier on the same TF–IDF features.
- Try an RBF kernel if interactions or a curved boundary are plausible.
- Compare both on the same validation data and inspect their errors.
- Better training results alone may mean the kernel fitted quirks of the training data.
- Sparse TF–IDF often works well with linear models; RBF SVC can be much slower as the number of documents grows.

Trees and Ensembles



Toy example, not a fitted classifier.

- A tree follows one path from feature tests to a label.
- A single tree can be unstable.
- Ensembles combine many trees (forest or boosting).
- For sparse TF–IDF text, compare with a linear baseline.

Modern Classifiers

- Modern text classification often uses:
  - pretrained embeddings
  - transformer encoders
  - fine-tuned BERT-style models
  - LLM prompting
- We return to these on Day 3.
- For now, the same logic remains:
  - labels
  - features / representations
  - train/test separation
  - validation
  - error analysis

Train–Validation–Test Split

Training Set	**Validation Set**	**Test Set**
learn parameters	tune model	final evaluation

The test set should not guide model development.

K-Fold Cross-Validation

- Useful when labeled data is limited or model choices are uncertain.
- Split data into k folds.
- For each fold:
  - train on $k-1$ folds
  - validate on the held-out fold
- Average performance across folds:

$$\widehat{M}_{CV} = \frac{1}{k} \sum_{j=1}^{k} M_j$$

- Use cross-validation for model selection; keep the test set for the final estimate.

K-Fold Cross-Validation: Mechanics

Run	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	Validate	Train	Train	Train	Train
2	Train	Validate	Train	Train	Train
3	Train	Train	Validate	Train	Train
4	Train	Train	Train	Validate	Train
5	Train	Train	Train	Train	Validate

- In imbalanced classification, use stratified folds so each fold has similar class proportions.
- Report the mean and the variation across folds, not only the best fold.
- Cross-validation reduces dependence on one lucky train-test split.

Bias–Variance Tradeoff

High bias	**High variance**
model is too simple	model is too sensitive to the training sample
misses real language patterns	learns accidental words or names
underfits training and test data	fits training data but fails on new data
example: narrow spam dictionary	example: overfit classifier on few labels

$$\mathbb{E}\big[(Y - \widehat{f}(X))^2\big] = \text{Bias}^2 + \text{Variance} + \text{Irreducible error}$$

Applied lesson

The goal is the model that generalizes best to unseen text.

Diagnosing Bias and Variance

Observed pattern	Likely issue	Possible response
Low train and low validation score	High bias	Better features, richer representation, less restrictive model
High train and low validation score	High variance	Regularization, simpler model, more labels, stricter validation
Large fold-to-fold variation	Unstable estimate	Stratified k-fold CV, inspect folds, collect broader labels

- Cross-validation estimates whether performance is stable.
- Regularization reduces variance by penalizing overly flexible models.

From Prediction to Evaluation

Why Evaluate?

- Are our models reliable?
- Are our models valid?
- Can we trust the resulting measurements?
- Evaluation allows:
  - comparison: is model A better than model B?
  - selection: which model should I use?
  - validation: can I trust these results?

Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	TP	FN
Actual Negative	FP	TN

Rows are the human labels; columns are the model's predictions.

Confusion Matrix: Interpretation

- **TP:** spam correctly flagged as spam
- **TN:** ordinary message correctly left in the inbox
- **FP:** ordinary message wrongly sent to spam
- **FN:** spam message wrongly left in the inbox

Important

False positives and false negatives may have different substantive costs.

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Intuitive and common.
- But can be misleading with imbalanced data.

Example

If 99% of documents are negative, a model that always predicts negative gets 99% accuracy.

Class Imbalance

- Many important text classification problems are imbalanced.
- Rare classes are often substantively important.
- Accuracy can hide total failure on minority classes.
- Always inspect:
  - class distribution
  - per-class precision
  - per-class recall
  - confusion matrix

Precision and Recall

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

- Precision: among predicted positives, how many are truly positive?
- Recall: among true positives, how many did we find?

Precision and Recall as Probabilities

- Let $y \in \{0, 1\}$ be the true class and $\widehat{y} \in \{0, 1\}$ the predicted class.

$$\text{Recall} = P(\widehat{y} = 1 \mid y = 1)$$

$$\text{Precision} = P(y = 1 \mid \widehat{y} = 1)$$

- Recall: probability of detecting a true positive.
- Precision: probability that a predicted positive is truly positive.

Different Error Costs

Application	**False positive cost**	**False negative cost**
Hate speech detection	unjust censorship	harmful content missed
Protest event detection	false event record	missing real protest
Self-harm detection	unnecessary intervention	serious case missed
Topic filtering	noisy corpus	relevant texts excluded

Metrics should follow the research and ethical stakes.

F1 Score

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Harmonic mean of precision and recall.
- Penalizes large imbalance.
- Useful when both false positives and false negatives matter.

F-beta Score

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

- $\beta > 1$: recall matters more
- $\beta < 1$: precision matters more
- Choose based on the task, not habit

Thresholds Are Decisions

Lower spam threshold	**Higher spam threshold**
flags more messages as spam	flags fewer messages as spam
higher recall	higher precision
more false positives	more false negatives
useful if missing spam is costly	useful if blocking ham is costly

Substantive point

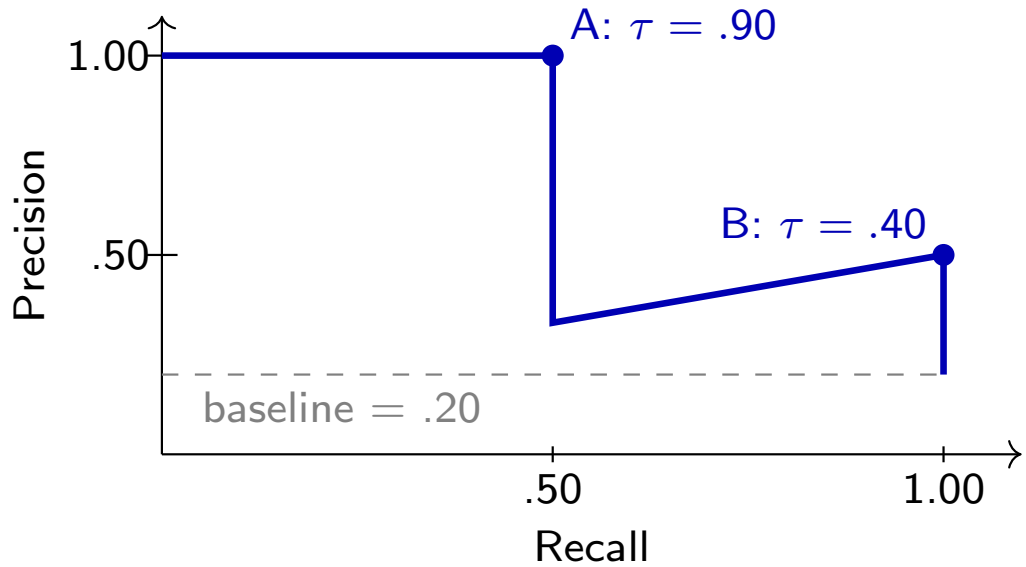
The best threshold depends on the cost of each error type. Check probability calibration before interpreting scores as risks.

Precision–Recall Curve: One Example

Ten messages: 2 spam, 8 ham. Top four scores, highest first:

$$.95\ (\text{spam}), \quad .80\ (\text{ham}), \quad .60\ (\text{ham}), \quad .40\ (\text{spam})$$

The other six ham messages score below .40.



Cutoff	**Precision**	**Recall**
A: cutoff .90; flag top 1	$1/1 = 1.00$	$1/2 = .50$
B: cutoff .40; flag top 4	$2/4 = .50$	$2/2 = 1.00$

Prediction Performance vs. Measurement Validity

- A classifier can predict labels well but still fail as a measure.
- It may learn:
  - topic shortcuts
  - source artifacts
  - annotator bias
  - temporal artifacts
  - language-specific patterns
  - corpus-specific signals
- Good performance is necessary, but not sufficient.

Error Analysis

- Do not only report metrics.
- Read false positives and false negatives.
- Ask:
  - Is the codebook unclear?
  - Is the model learning shortcuts?
  - Are some topics harder?
  - Are some sources harder?
  - Are some groups or languages harder?
  - Are errors substantively meaningful?

Validation Is Not Just Accuracy

- Does the model match human labels?
- Are the human labels valid?
- Does the model generalize across:
  - time
  - sources
  - topics
  - countries
  - languages
  - platforms
- Do the predictions support the substantive claim?

Practical Session

First together, then your own two changes

Guided Walkthrough: We Do This Together

Open `day_2_labels_and_classification.ipynb`.

1. Run the setup cells, then go to the **SMS spam** section.
2. Inspect labels, apply the dictionary, and fit the supplied classifiers.
3. Compare metrics and read one prediction error.
4. Compare linear and RBF SVM on the small validation example.

This is the demonstration; the next notebook is your exercise.

Student Exercise: Two Code Changes

Open `day_2_spam_exercise.ipynb`. Data, split, and baselines are supplied.

1. **Dictionary:** add at least two words, remove one, and apply the new rule.
2. **TF–IDF:** change either `min_df` or the n-gram range; refit and predict.
3. **Compare:** baseline vs. edited metrics; read false positives and negatives.

Submit: four or five sentences on what changed, why, and which version you trust.

Key Takeaways

- Concepts need operationalization.
- Dictionaries are transparent rule-based measurement instruments.
- Regex and search strings are useful but brittle.
- Codebooks map theoretical concepts onto labels.
- Supervised ML learns from labels, not from theory directly.
- Evaluation metrics answer different questions.
- Error analysis is often more informative than a single score.
- Validation requires connecting predictions back to the construct.

The Main Lesson

A classifier is not automatically a measurement instrument.

It becomes one only when the labels, model, errors, and validation strategy are connected back to the concept.

Unsupervised Machine Learning

Supervised and unsupervised learning

- Supervised learning uses labelled examples to predict an outcome.
- Unsupervised learning describes patterns in data without training labels.
- The same texts may support different groupings or dimensions, depending on the representation and model.

One target: a parameter vector

In a model-based account, a vector θ describes a particular feature of the world that we want to learn about.

$$\text{world and data-generating process} \longrightarrow \theta \longrightarrow \hat{\theta}$$

- θ is the target; $\hat{\theta}$ is our estimate from data.
- The target is defined by the question and model. Calling it a single target does not mean the whole of reality has one description.

Approximating a target

If the target θ is well defined, we can ask how close an estimate comes:

$$L(\theta, \hat{\theta}) = \|\theta - \hat{\theta}\|^2.$$

- Parameters might describe how features relate to a specified outcome.
- Observed labels let us assess predictions on held-out examples.
- Predictive accuracy and recovery of the true parameters are different goals; one does not guarantee the other.

What changes without labels?

- With a specified label, we can compare predicted and observed outcomes.
- In an exploratory task, there may be no predefined label or agreed target structure to recover.
- A model instead finds a structure $\hat{S}$ in observed texts X according to its objective.

$$X \longrightarrow \hat{S}$$

The same texts, different structures

Consider a collection of parliamentary speeches. We could organize them by:

- **Policy area:** economy, defence, education.
- **Political position:** government versus opposition, or a left-right dimension.
- **Style:** procedural language versus emotional appeals.

These structures can coexist. Each answers a different question and may require different features and methods.

A single correct organization?

- Assuming one true θ for the entire collection can suggest there is only one correct way to organize it.
- Unsupervised methods can uncover several meaningful structures rather than approximating one universally correct classification.
- A specific model can still assume a particular latent structure and estimate its parameters.

What makes a discovered structure meaningful?

- Does it answer a useful substantive question?
- Do representative texts support the proposed interpretation?
- Does it remain recognizable across reasonable model and preprocessing choices?
- Does relevant external evidence support the interpretation?

Different structures can be useful, but not every grouping is equally convincing.

Measurement and model objectives

An objective function describes what a model optimizes. For example, k-means seeks compact clusters.

- A research question gives those patterns a possible interpretation.
- A good optimization score alone cannot establish that a cluster or topic measures a social concept.
- Computing performance and substantive interpretation matter in both computer science and social science.

Discovery and interpretation

- Models can reveal unexpected patterns as well as examine theoretically motivated questions.
- A topic or cluster is a result to investigate, not automatically a real-world category.
- Examine representative documents and check stability, alternative explanations, and relevant external evidence.

What is Measurement?

- Measurement assigns numbers or symbols to phenomena according to rules.
- It involves three linked steps:
  - **Conceptualization:** What do we want to capture?
  - **Operationalization:** How can we represent it empirically?
  - **Quantification:** How do we express it numerically?
- Measurement maps theoretical constructs to observed data.

What Is an Objective Function?

- The **objective function** defines what a model tries to achieve.
- It translates a goal into something computable.
- Examples:
  - Linear regression: minimize squared error $(y - \hat{y})^2$.
  - K-means: minimize within-cluster variance.
  - PCA: minimize reconstruction error.
- It is the model's definition of success.

Why the Objective Function Matters

- Defines what the model cares about.
- Different objectives $\Rightarrow$ different structures discovered.
- If the objective does not align with theory, results can mislead.

Good measurement = alignment between objective and theory.

Prediction vs Inference

- **Prediction:** How well does the model reproduce unseen data?
- **Inference:** What do parameters tell us about the world?
- Large parameter spaces can yield good prediction but poor inference.

Problems

- **Translating concepts:** machine learning measures patterns, not constructs.
- **Connecting methods to theory:** models optimize statistical objectives, not theoretical meanings.
- **Understanding measurement:** without a clear theory-to-model mapping, $\hat{S}$ may lack an interpretable referent.

Families of Unsupervised Methods

- **Clustering:** groups of similar items. Examples: K-means, hierarchical clustering.
- **Topic modeling:** latent themes. Examples: LDA, Structural Topic Model (STM).
- **Scaling:** latent dimensions. Example: Wordfish (Slapin & Proksch, 2008).

K-Means Clustering

- Partitions data into K non-overlapping clusters.
- A simple(ish) algorithmic method.

K-Means Clustering

$$C_1, C_2, \ldots, C_K$$

K-Means Clustering

$$C_1, C_2, \ldots, C_K$$

$$C_1 \cup C_2 \cup \cdots \cup C_K = \{1, \ldots, n\}$$

K-Means Clustering

$$C_1, C_2, \ldots, C_K$$

$$C_1 \cup C_2 \cup \cdots \cup C_K = \{1, \ldots, n\}$$

$$C_k \cap C_{k'} = \varnothing \quad \text{for all } k \neq k'$$

K-Means Clustering

$$C_1, C_2, \ldots, C_K$$

$$C_1 \cup C_2 \cup \cdots \cup C_K = \{1, \ldots, n\}$$

$$C_k \cap C_{k'} = \varnothing \quad \text{for all } k \neq k'$$

Each observation belongs to exactly one cluster.
Clusters are non-overlapping and collectively exhaustive.

Objective Function

Goal: Find K clusters that minimize within-cluster variation.

Objective Function

Goal: Find K clusters that minimize within-cluster variation.

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

Objective Function

Goal: Find K clusters that minimize within-cluster variation.

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid (mean) of cluster C_k:

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$$

Objective Function

Goal: Find K clusters that minimize within-cluster variation.

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid (mean) of cluster C_k:

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$$

Intuition: assign points to clusters so that each cluster is as compact as possible.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

1. Choose the number of clusters K.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

1. Choose the number of clusters K.
2. Randomly assign each observation to one of the K clusters.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

1. Choose the number of clusters K.
2. Randomly assign each observation to one of the K clusters.
3. Compute each centroid: $\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

1. Choose the number of clusters K.
2. Randomly assign each observation to one of the K clusters.
3. Compute each centroid: $\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$.
4. Reassign each observation to the nearest centroid.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

1. Choose the number of clusters K.
2. Randomly assign each observation to one of the K clusters.
3. Compute each centroid: $\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$.
4. Reassign each observation to the nearest centroid.
5. Repeat steps 3–4 until assignments do not change.

K-Means Algorithm

Algorithm: Iterative refinement of cluster assignments.

1. Choose the number of clusters K.
2. Randomly assign each observation to one of the K clusters.
3. Compute each centroid: $\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$.
4. Reassign each observation to the nearest centroid.
5. Repeat steps 3–4 until assignments do not change.

K-means converges to a local optimum, not necessarily the global one.

Choosing the Number of Clusters

- **Elbow method:** plot within-cluster sum of squares against K.
- **Silhouette score:** compare distance to own cluster with distance to others.
- **Theory-informed:** choose K based on expected structure.

Cluster Algorithms Validation

- **Data assumptions:** do clustering assumptions match data generation?

Cluster Algorithms Validation

- **Data assumptions:** do clustering assumptions match data generation?
- **Internal validity:** cohesion and separation.

Cluster Algorithms Validation

- **Data assumptions:** do clustering assumptions match data generation?
- **Internal validity:** cohesion and separation.
- **External validity:** alignment with existing understanding or labels.

Cluster Algorithms Validation

- **Data assumptions:** do clustering assumptions match data generation?
- **Internal validity:** cohesion and separation.
- **External validity:** alignment with existing understanding or labels.
- **Cross-validity:** stability across similar datasets or time periods.

Cluster Algorithms Validation

- **Data assumptions:** do clustering assumptions match data generation?
- **Internal validity:** cohesion and separation.
- **External validity:** alignment with existing understanding or labels.
- **Cross-validity:** stability across similar datasets or time periods.

Interpretation and theory provide the ultimate validation.

Topic Models

Topic Modeling

- **Goal:** discover latent themes or topics in documents.

Topic Modeling

- **Goal:** discover latent themes or topics in documents.
- Topic modeling is a **family of models**, not a single algorithm.

Topic Modeling

- **Goal:** discover latent themes or topics in documents.
- Topic modeling is a **family of models**, not a single algorithm.
- **Latent Dirichlet Allocation (LDA)** is one topic model.

Topic Modeling

- **Goal:** discover latent themes or topics in documents.
- Topic modeling is a **family of models**, not a single algorithm.
- **Latent Dirichlet Allocation (LDA)** is one topic model.
- Other approaches include LSA, SVD, and clustering-based topic discovery.

Latent Dirichlet Allocation (LDA)

- A **Bayesian generative hierarchical model** for discovering topics in text.

Latent Dirichlet Allocation (LDA)

- A **Bayesian generative hierarchical model** for discovering topics in text.
- Introduced to model population structure in genetics (Pritchard, Stephens & Donnelly, 2000).

Latent Dirichlet Allocation (LDA)

- A **Bayesian generative hierarchical model** for discovering topics in text.
- Introduced to model population structure in genetics (Pritchard, Stephens & Donnelly, 2000).
- Adapted to text analysis (Blei, Ng & Jordan, 2003).

Latent Dirichlet Allocation (LDA)

- A **Bayesian generative hierarchical model** for discovering topics in text.
- Introduced to model population structure in genetics (Pritchard, Stephens & Donnelly, 2000).
- Adapted to text analysis (Blei, Ng & Jordan, 2003).
- Each document is a *mixture of topics*; each topic is a distribution over words.

Latent Dirichlet Allocation (LDA)

- **Goal:** discover latent topics that generate observed words.
- **Assumption:** documents are mixtures of topics; topics are distributions over words.

Mixture Models

- Observations come from a combination of underlying distributions.

Mixture Models

- Observations come from a combination of underlying distributions.
- Each distribution represents a **latent group or component**.

Mixture Models

- Observations come from a combination of underlying distributions.
- Each distribution represents a **latent group or component**.
- Component probabilities are weighted by mixture proportions.

Mixture Models

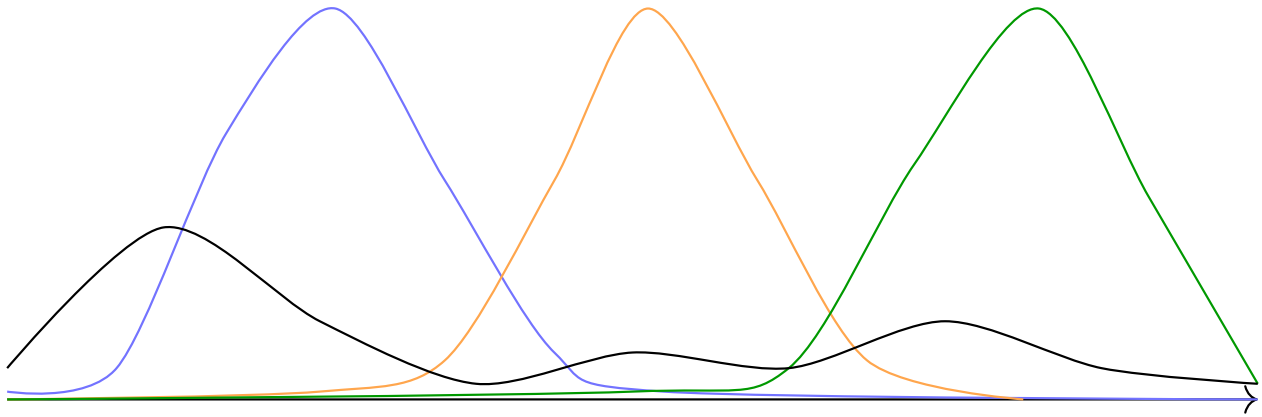
- Observations come from a combination of underlying distributions.
- Each distribution represents a **latent group or component**.
- Component probabilities are weighted by mixture proportions.
- Unlike K-means, mixture models use *soft assignment*.

Mixture Models

- Observations come from a combination of underlying distributions.
- Each distribution represents a **latent group or component**.
- Component probabilities are weighted by mixture proportions.
- Unlike K-means, mixture models use *soft assignment*.

Example: a document can be 70% about politics, 20% about health, and 10% about sports.

Mixture Model



Observed data = weighted combination of latent distributions.

Hierarchical Models

$$y = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k + \varepsilon$$

Hierarchical Models

$$y = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k + \varepsilon$$

In a standard regression model, coefficients β are fixed unknowns. We estimate them directly from the data.

Hierarchical Models

$$y = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k + \varepsilon$$

In a standard regression model, coefficients β are fixed unknowns. We estimate them directly from the data. In hierarchical (Bayesian) models,

coefficients are random variables with their own probability distributions.

Hierarchical Models

$$
\begin{aligned}
y &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k \\
\beta &\sim \text{Normal}(0, 5) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

Hierarchical Models

$$
\begin{aligned}
y &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k \\
\beta &\sim \text{Normal}(0, 5) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

- **Likelihood:** how data are generated.
- **Priors:** β and σ have their own distributions.
- Parameters depend on other parameters.

Hierarchical Models: Hyperparameters

$$\beta \sim \text{Normal}(0, 5), \qquad \sigma \sim \text{Exponential}(1)$$

Hierarchical Models: Hyperparameters

$$\beta \sim \text{Normal}(0, 5), \qquad \sigma \sim \text{Exponential}(1)$$

The numbers $(0, 5)$ and (1) are **hyperparameters**: they define the prior distributions for model parameters.

Hierarchical Models: Hyperparameters

$$\beta \sim \text{Normal}(0, 5), \qquad \sigma \sim \text{Exponential}(1)$$

The numbers $(0, 5)$ and (1) are **hyperparameters**: they define the prior distributions for model parameters. In LDA, Dirichlet priors α and β play the same role. They control how concentrated or diffuse topic and document distributions are.

LDA Formalized

Generative process:

1. For each topic $k \in \{1, \ldots, K\}$:

$$\theta_k \sim \text{Dirichlet}(\alpha) \qquad \text{(words in topic } k\text{)}$$

LDA Formalized

Generative process:

1. For each topic $k \in \{1, \ldots, K\}$:

$$\theta_k \sim \text{Dirichlet}(\alpha) \qquad \text{(words in topic } k\text{)}$$

2. For each document $d \in \{1, \ldots, D\}$:

$$\eta_d \sim \text{Dirichlet}(\beta) \qquad \text{(topics in document } d\text{)}$$

LDA Formalized

Generative process:

1. For each topic $k \in \{1, \ldots, K\}$:

$$\theta_k \sim \text{Dirichlet}(\alpha) \qquad \text{(words in topic } k\text{)}$$

2. For each document $d \in \{1, \ldots, D\}$:

$$\eta_d \sim \text{Dirichlet}(\beta) \qquad \text{(topics in document } d\text{)}$$

3. For each word w in document d:

$$z_{d,w} \sim \text{Multinomial}(\eta_d), \qquad x_{d,w} \sim \text{Multinomial}(\theta_{z_{d,w}})$$

LDA Formalized

Generative process:

1. For each topic $k \in \{1, \ldots, K\}$:

$$\theta_k \sim \text{Dirichlet}(\alpha) \qquad \text{(words in topic } k\text{)}$$

2. For each document $d \in \{1, \ldots, D\}$:

$$\eta_d \sim \text{Dirichlet}(\beta) \qquad \text{(topics in document } d\text{)}$$

3. For each word w in document d:

$$z_{d,w} \sim \text{Multinomial}(\eta_d), \qquad x_{d,w} \sim \text{Multinomial}(\theta_{z_{d,w}})$$

Goal: infer latent θ, η, z from observed words x.

LDA Formalized

Generative process:

1. For each topic $k \in \{1, \ldots, K\}$:

$$\theta_k \sim \text{Dirichlet}(\alpha) \qquad \text{(words in topic } k\text{)}$$

2. For each document $d \in \{1, \ldots, D\}$:

$$\eta_d \sim \text{Dirichlet}(\beta) \qquad \text{(topics in document } d\text{)}$$

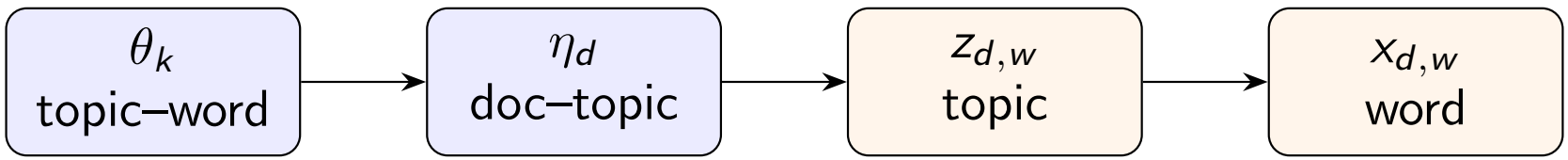
3. For each word w in document d:

$$z_{d,w} \sim \text{Multinomial}(\eta_d), \qquad x_{d,w} \sim \text{Multinomial}(\theta_{z_{d,w}})$$

Goal: infer latent θ, η, z from observed words x.

$$P(\theta, \eta, z \mid x, \alpha, \beta)$$

LDA Generative Process



Model first draws topic–word distributions, then document–topic mixtures, assigns topics to words, and generates observed words.

LDA in Words

1. For each topic k, draw word distribution $\theta_k \sim \text{Dirichlet}(\alpha)$.
2. For each document d, draw topic mixture $\eta_d \sim \text{Dirichlet}(\beta)$.
3. For each word:
   - Choose topic $z_{d,w} \sim \text{Multinomial}(\eta_d)$.
   - Choose word $x_{d,w} \sim \text{Multinomial}(\theta_{z_{d,w}})$.

Dirichlet Distribution

- A distribution over distributions.

Dirichlet Distribution

- A distribution over distributions.
- It describes random vectors that sum to one:

$$\theta = (\theta_1, \ldots, \theta_K), \qquad \sum_{k=1}^{K} \theta_k = 1.$$

Dirichlet Distribution

- A distribution over distributions.
- It describes random vectors that sum to one:

$$\theta = (\theta_1, \ldots, \theta_K), \qquad \sum_{k=1}^{K} \theta_k = 1.$$

- Parameterized by $\alpha = (\alpha_1, \ldots, \alpha_K)$, which controls concentration.

Dirichlet Distribution

- A distribution over distributions.
- It describes random vectors that sum to one:

$$\theta = (\theta_1, \ldots, \theta_K), \qquad \sum_{k=1}^{K} \theta_k = 1.$$

- Parameterized by $\alpha = (\alpha_1, \ldots, \alpha_K)$, which controls concentration.
- High α: similar probabilities (uniform). Low α: few dominate (sparse).

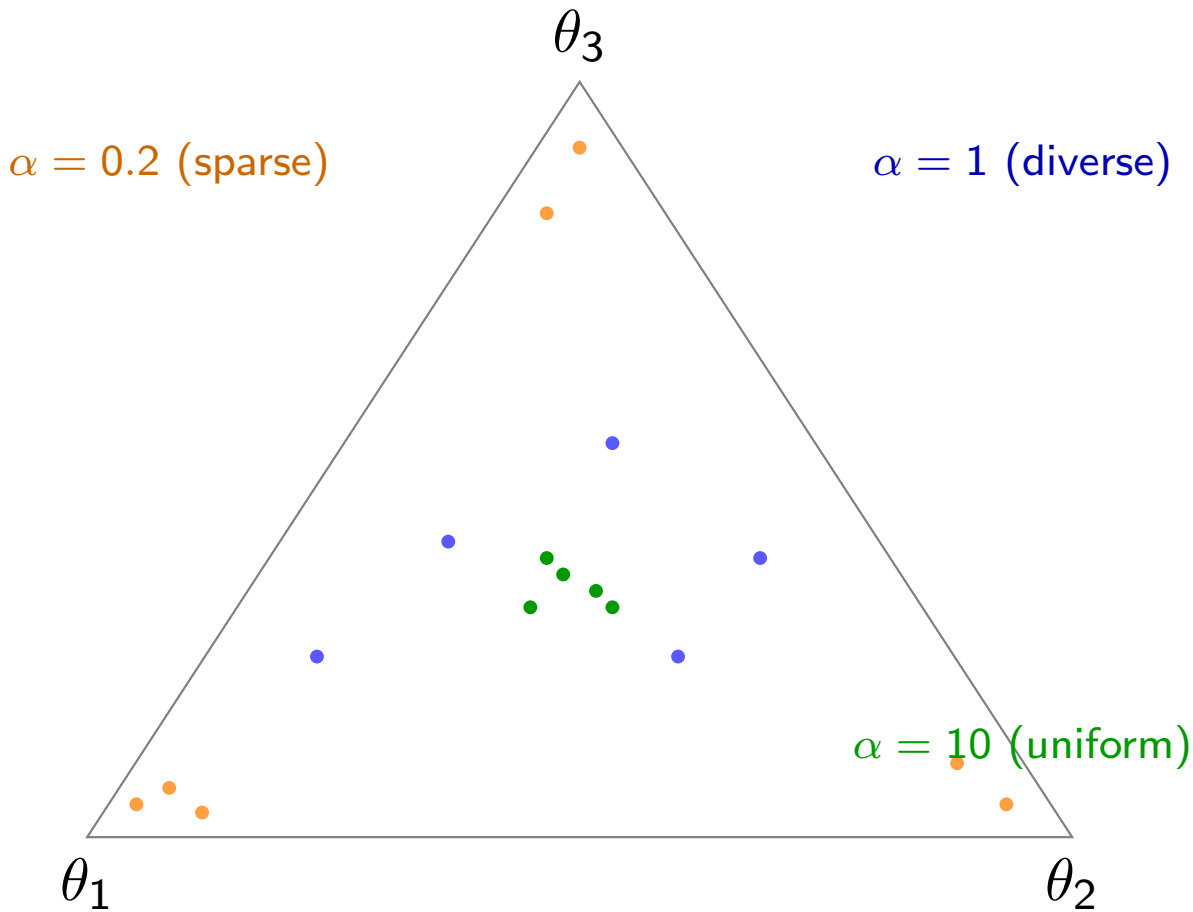
Dirichlet Distribution

- A distribution over distributions.
- It describes random vectors that sum to one:

$$\theta = (\theta_1, \ldots, \theta_K), \qquad \sum_{k=1}^{K} \theta_k = 1.$$

- Parameterized by $\alpha = (\alpha_1, \ldots, \alpha_K)$, which controls concentration.
- High α: similar probabilities (uniform). Low α: few dominate (sparse).
- In LDA, Dirichlet priors control the concentration of topics or words.

Visualizing the Dirichlet Distribution



Smaller α gives more peaked draws; larger α gives more uniform draws. Each dot is a draw $(\theta_1, \theta_2, \theta_3)$ on the simplex.

Dirichlet in LDA

Variable	Drawn from	Meaning
η_d	Dirichlet(β)	Topic distribution for document d
θ_k	Dirichlet(α)	Word distribution for topic k

Dirichlet priors define the diversity or sparsity of topics and words.

Dirichlet priors in this notation

In the notation used on the preceding slides:

- $\eta_d \sim$ Dirichlet(β): document d's distribution over topics.
- $\theta_k \sim$ Dirichlet(α): topic k's distribution over words.

For symmetric priors with a fixed number of categories, smaller values tend to produce sparser draws and larger values more even draws.

Notation varies: many references use α for document–topic and β for topic–word priors. Check the software or paper.

From LDA to STM

- **LDA:** all documents share the same prior over topics.

From LDA to STM

- **LDA:** all documents share the same prior over topics.
- **STM (Structural Topic Model):** topic prevalence and content vary with document covariates.

From LDA to STM

- **LDA:** all documents share the same prior over topics.
- **STM (Structural Topic Model):** topic prevalence and content vary with document covariates.
- Covariates can affect:
  - Topic *prevalence*: how much each topic appears in a document.
  - Topic *content*: which words discuss that topic.

From LDA to STM

- **LDA:** all documents share the same prior over topics.
- **STM (Structural Topic Model):** topic prevalence and content vary with document covariates.
- Covariates can affect:
  - Topic *prevalence*: how much each topic appears in a document.
  - Topic *content*: which words discuss that topic.
- STM uses a logistic-normal prior:

$$\eta_d \sim \text{LogisticNormal}(\mu_d, \Sigma), \qquad \mu_d = X_d\gamma.$$

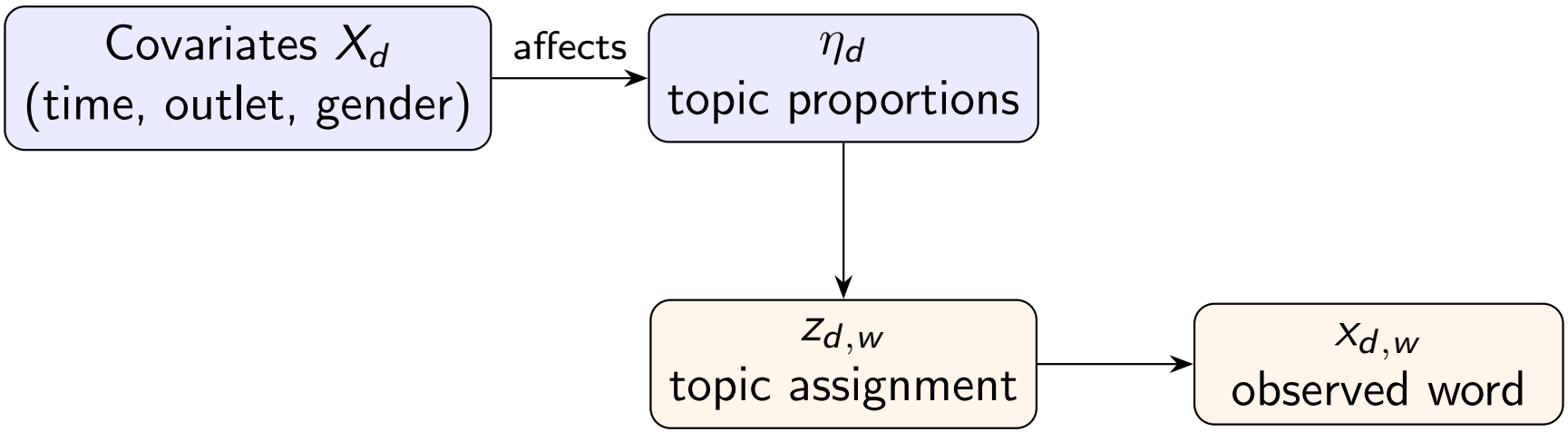
From LDA to STM

- **LDA:** all documents share the same prior over topics.
- **STM (Structural Topic Model):** topic prevalence and content vary with document covariates.
- Covariates can affect:
  - Topic *prevalence*: how much each topic appears in a document.
  - Topic *content*: which words discuss that topic.
- STM uses a logistic-normal prior:

$$\eta_d \sim \text{LogisticNormal}(\mu_d, \Sigma), \qquad \mu_d = X_d\gamma.$$

- This permits inference about topics by time, source, gender, ideology, and other covariates.

How STM Extends LDA



STM adds document-level structure: covariates influence which topics appear (*prevalence*) and how topics are expressed (*content*).

Why Validate Unsupervised Models?

- No ground truth $\Rightarrow$ a model can always find patterns.

Why Validate Unsupervised Models?

- No ground truth $\Rightarrow$ a model can always find patterns.
- Validation checks whether discovered structure reflects the data (internal coherence), world (external correspondence), or theory (interpretive meaning).

Why Validate Unsupervised Models?

- No ground truth $\Rightarrow$ a model can always find patterns.
- Validation checks whether discovered structure reflects the data (internal coherence), world (external correspondence), or theory (interpretive meaning).
- Without validation, unsupervised results may be artifacts of the algorithm.

Validation of Unsupervised Models

- **Data assumptions:** do model assumptions match the data structure? K-means assumes spherical clusters; LDA assumes word exchangeability.

Validation of Unsupervised Models

- **Data assumptions:** do model assumptions match the data structure? K-means assumes spherical clusters; LDA assumes word exchangeability.
- **Internal validity:** cohesion, silhouette, WCSS, and topic coherence.

Validation of Unsupervised Models

- **Data assumptions:** do model assumptions match the data structure? K-means assumes spherical clusters; LDA assumes word exchangeability.
- **Internal validity:** cohesion, silhouette, WCSS, and topic coherence.
- **External validity:** compare with known classifications, human judgments, or metadata.

Validation of Unsupervised Models

- **Data assumptions:** do model assumptions match the data structure? K-means assumes spherical clusters; LDA assumes word exchangeability.
- **Internal validity:** cohesion, silhouette, WCSS, and topic coherence.
- **External validity:** compare with known classifications, human judgments, or metadata.
- **Cross-validity:** stability across samples, periods, and model runs.

Validation of Unsupervised Models

- **Data assumptions:** do model assumptions match the data structure? K-means assumes spherical clusters; LDA assumes word exchangeability.
- **Internal validity:** cohesion, silhouette, WCSS, and topic coherence.
- **External validity:** compare with known classifications, human judgments, or metadata.
- **Cross-validity:** stability across samples, periods, and model runs.
- **Interpretive validity:** qualitative assessment of meaning by the researcher.

Validating Topic Models

- **Semantic validity:** do top words within each topic form a coherent theme? Check with human evaluation.

Validating Topic Models

- **Semantic validity:** do top words within each topic form a coherent theme? Check with human evaluation.
- **Convergent validity:** do topic proportions correlate with external variables or manual codings?

Validating Topic Models

- **Semantic validity:** do top words within each topic form a coherent theme? Check with human evaluation.
- **Convergent validity:** do topic proportions correlate with external variables or manual codings?
- **Quantitative metrics:** topic coherence (NPMI, UMass, UCI) and held-out likelihood or perplexity.

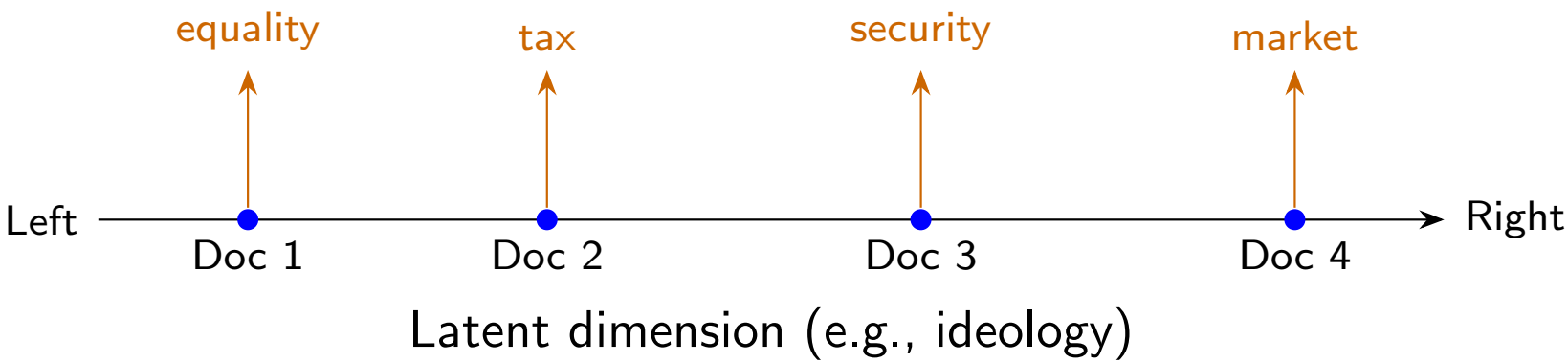
Validating Topic Models

- **Semantic validity:** do top words within each topic form a coherent theme? Check with human evaluation.
- **Convergent validity:** do topic proportions correlate with external variables or manual codings?
- **Quantitative metrics:** topic coherence (NPMI, UMass, UCI) and held-out likelihood or perplexity.
- **Human-in-the-loop validation:** experts or coders rate interpretability and label consistency.

Text Scaling Methods

- Place documents on latent dimensions (e.g., ideology).
- Use discriminating word frequencies as anchors.

Text Scaling



Documents are positioned along a latent dimension according to word usage. Discriminating words push them toward one end or the other.

Assumptions of Text Scaling Methods

- **Unidimensional structure:** texts vary along one latent continuum.

Assumptions of Text Scaling Methods

- **Unidimensional structure:** texts vary along one latent continuum.
- **Conditional independence:** words are independent given a document's latent position.

Assumptions of Text Scaling Methods

- **Unidimensional structure:** texts vary along one latent continuum.
- **Conditional independence:** words are independent given a document's latent position.
- **Stable word meanings:** each word has the same directional association across texts.

Assumptions of Text Scaling Methods

- **Unidimensional structure:** texts vary along one latent continuum.
- **Conditional independence:** words are independent given a document's latent position.
- **Stable word meanings:** each word has the same directional association across texts.
- **Comparability:** all documents come from the same domain or context.

Assumptions of Text Scaling Methods

- **Unidimensional structure:** texts vary along one latent continuum.
- **Conditional independence:** words are independent given a document's latent position.
- **Stable word meanings:** each word has the same directional association across texts.
- **Comparability:** all documents come from the same domain or context.
- **Signal vs. topic:** word use reflects position, not topic differences.

Assumptions of Text Scaling Methods

- **Unidimensional structure:** texts vary along one latent continuum.
- **Conditional independence:** words are independent given a document's latent position.
- **Stable word meanings:** each word has the same directional association across texts.
- **Comparability:** all documents come from the same domain or context.
- **Signal vs. topic:** word use reflects position, not topic differences.
- **Scale invariance:** only relative positions matter; the origin is arbitrary.

Wordfish as a Hierarchical Model (Slapin & Proksch, 2008)

Likelihood:

$$y_{ik} \sim \text{Poisson}(\lambda_{ik}), \qquad \log \lambda_{ik} = \alpha_i + \psi_k + \beta_k \omega_i.$$

Wordfish as a Hierarchical Model (Slapin & Proksch, 2008)

Likelihood:

$$y_{ik} \sim \text{Poisson}(\lambda_{ik}), \qquad \log \lambda_{ik} = \alpha_i + \psi_k + \beta_k \omega_i.$$

Parameters:

- α_i: document-specific intercept (length / verbosity).
- ψ_k: word-specific intercept (baseline frequency).
- β_k: word-specific discrimination.
- ω_i: document position on the latent scale.

Wordfish as a Hierarchical Model (Slapin & Proksch, 2008)

Likelihood:

$$y_{ik} \sim \text{Poisson}(\lambda_{ik}), \qquad \log \lambda_{ik} = \alpha_i + \psi_k + \beta_k \omega_i.$$

Parameters:

- α_i: document-specific intercept (length / verbosity).
- ψ_k: word-specific intercept (baseline frequency).
- β_k: word-specific discrimination.
- ω_i: document position on the latent scale.

Hierarchical structure:

$$\alpha_i \sim \text{Normal}(0, \sigma_\alpha), \quad \psi_k \sim \text{Normal}(0, \sigma_\psi), \quad \beta_k \sim \text{Normal}(0, \sigma_\beta), \quad \omega_i \sim \text{Normal}(0, 1).$$

Wordfish as a Hierarchical Model (Slapin & Proksch, 2008)

Likelihood:

$$y_{ik} \sim \text{Poisson}(\lambda_{ik}), \qquad \log \lambda_{ik} = \alpha_i + \psi_k + \beta_k \omega_i.$$

Parameters:

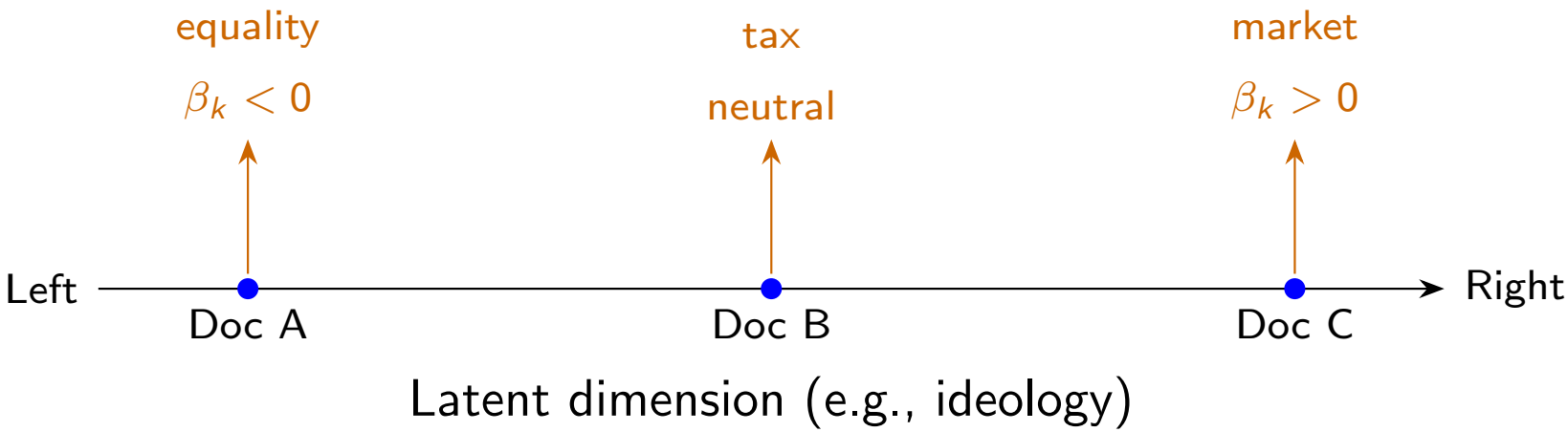
- α_i: document-specific intercept (length / verbosity).
- ψ_k: word-specific intercept (baseline frequency).
- β_k: word-specific discrimination.
- ω_i: document position on the latent scale.

Hierarchical structure:

$$\alpha_i \sim \text{Normal}(0, \sigma_\alpha), \quad \psi_k \sim \text{Normal}(0, \sigma_\psi), \quad \beta_k \sim \text{Normal}(0, \sigma_\beta), \quad \omega_i \sim \text{Normal}(0, 1).$$

Wordfish estimates document positions ω_i and word discriminations β_k jointly.

How Wordfish Links Words and Documents



Words with positive β_k pull documents right; negative β_k pull them left.
Estimated ω_i summarize these tendencies.

Summary: Comparing Unsupervised Methods

Method	Goal	Output	Example
Clustering	Grouping	Cluster labels	K-means
Topic modeling	Thematic discovery	Topic-word probabilities	LDA / STM
Scaling	Latent dimension	Position scores	Wordfish

What Unsupervised Learning Does (and Doesn't Do)

- Discovers structure; it does not define it.
- Measures relationships, not truth.
- Depends on assumptions about meaning, similarity, and latent space.
- Useful for exploration and theory building, not just prediction.

Unsupervised learning helps us see structure we did not know was there, but we must interpret what it means.

Each method is a different kind of measurement, and every measurement is a theoretical exercise.

Questions?